

Unit - 7 : Exceptions

Topic : Errors, Runtime Errors

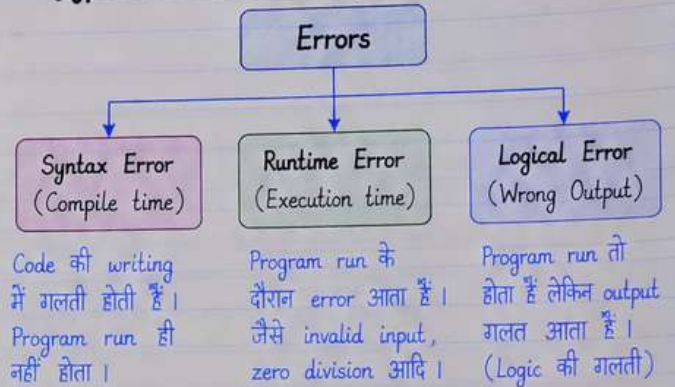
- Python program को run करते समय कभी-कभी errors (तुटियाँ) आती हैं, जिनके कारण program सही से execute नहीं हो पाता।
- Errors के different types होते हैं जैसे Syntax Error, Runtime Error और Logical Error।
- Runtime Errors वे errors होते हैं जो program run होने के दौरान (execution time पर) occur होते हैं।
- इन errors को handle करने के लिए Python में Exception Handling का use किया जाता है।

1. What is an Error ?

- Error एक ऐसी स्थिति होती है जिसमें program अपेक्षित रूप से काम नहीं कर पाता।
- यह code में किसी गलती (mistake) या गलत input के कारण होता है।
- Python में error आने पर interpreter एक error message दिखाता है जिसे traceback कहते हैं।

Note : सभी errors programs को रोक देते हैं, जब तक उनको handle न किया जाए।

2. Types of Errors in Python



3. Syntax Error

- जब code Python के syntax rules के अनुसार नहीं लिखा होता, तो Syntax Error आता है।
- यह compile time पर detect हो जाता है, इसलिए program run नहीं होता।
- Interpreter तुरंत error message दिखाता है।

Example :

```
print("Hello"),  
# SyntaxError: unterminated string literal
```

4. Runtime Error

- Runtime Error program के execution के दौरान आता है।
- यह तब होता है जब program syntactically correct होता है, लेकिन run-time पर कोई invalid operation किया जाता है।
- जैसे किसी संख्या को 0 से divide करना, invalid input देना, file न मिलना आदि।

Example :

```
a = 10  
b = 0  
print(a / b) # ZeroDivisionError: division by zero
```

5. Logical Error

- Logical Error में code का syntax सही होता है और program run भी हो जाता है, लेकिन output अपेक्षित (expected) नहीं होता है।
- यह programmer की logic में गलती के कारण होता है।

Example :

```
a = 5  
b = 3  
print(a + b) # यदि हमें multiplication चाहिए था  
# तो यह logic error है (output गलत)
```

6. Common Runtime Errors

Error Type	Description
ZeroDivisionError	किसी संख्या को 0 से divide करने पर
TypeError	गलत data type के operation पर
ValueError	Invalid value देने पर (जैसे int("abc"))
IndexError	List के index range से बाहर access करने पर
KeyError	Dictionary में non-existent key access करने पर
FileNotFoundError	File न मिलने पर
NameError	Undefined variable के use करने पर
AttributeError	Invalid attribute access करने पर

7. Runtime Error Example (Different Types)

ValueError

```
n = int("abc")  
# ValueError: invalid literal  
# for int() with base 10
```

TypeError

```
a = 5  
print(a + "hello")  
# TypeError: unsupported  
# operand type(s)
```

IndexError

```
lst = [1, 2, 3]  
print(lst[5])  
# IndexError: list index  
# out of range
```

FileNotFoundError

```
f = open("data.txt")  
# FileNotFoundError:  
# [Errno 2] No such file
```

8. Key Points

- Errors program का normal flow रोक देते हैं।
- Runtime Errors execution time पर occur होते हैं।
- हर error के लिए Python एक specific exception class देता है।
- Error message (traceback) से error का कारण पता चलता है।
- इन errors को handle करने के लिए try-except block का use किया जाता है (यह अगले topic में पढ़ेंगे)।

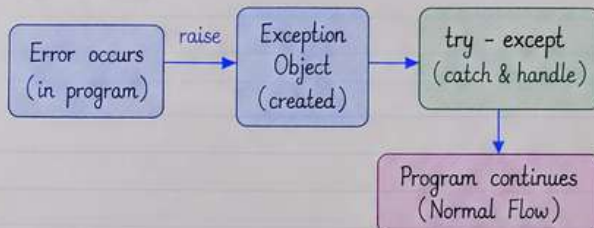
Topic : The Exception Model , Exception Hierarchy

- Python में Exception एक event होता है जो program के execution के दौरान occur होता है, जब कोई error आती है और normal flow of program break हो जाता है।
- Exception Model यह define करता है कि exception को कैसे handle किया जाता है।
- Exception Hierarchy, Python में built-in exception classes की एक hierarchical structure है।
- यह structure हमें different types की exceptions को understand और handle करने में मदद करती है।

1. The Exception Model

- Exception Model बताता है कि जब program में कोई error होती है, तब Python उस error को एक Exception object के रूप में create करता है और उसे raise करता है।
- यह exception object में error से related information (जैसे error type, message, stack trace) होती है।
- Program में try-except blocks के द्वारा इन exceptions को catch और handle किया जाता है ताकि program abnormally terminate न हो।

Exception Flow



2. How Exception Handling Works?

- ① Error occur होती है (जैसे invalid input, file not found आदि)।
- ② Python उस error के लिए appropriate exception class का object create करता है।
- ③ यह exception object को raise करता है।
- ④ Program try block में उस exception को catch करता है। (अगर matching except block है)।
- ⑤ Exception handle होने के बाद program normal flow में continue करता है।

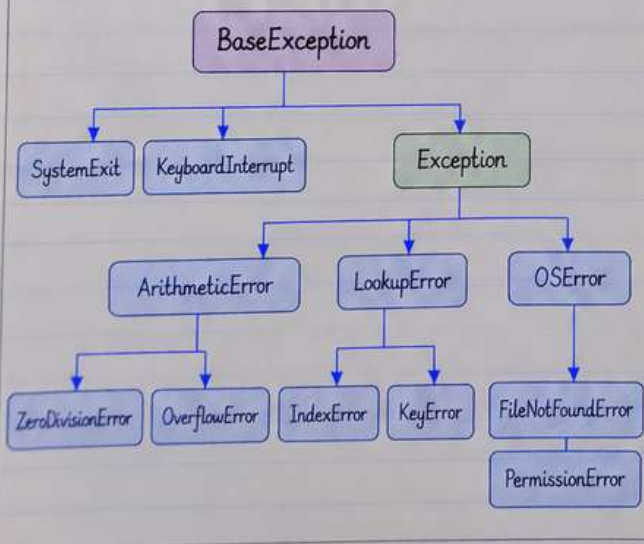
Example :

```
try:                                     # try block
    a = 10
    b = 0
    print(a / b)                         # error (division by zero)
except ZeroDivisionError as e:          # specific exception handle
    print("Error:", e)                  # error message print
print("Program continues...")          # यह line execute होगी
```

3. Exception Hierarchy

- Python में सभी built-in exceptions एक class hierarchy में organized होती हैं।
- Base class "BaseException" होती है, जो सभी exceptions की parent class है।
- "Exception" class, BaseException की subclass है और अधिकतर built-in exceptions इसी के under आती हैं।
- इस hierarchy की मदद से हम general या specific exceptions को easily handle कर सकते हैं।

Exception Hierarchy (Simplified View)



4. Some Common Exceptions

Exception Class	Meaning / When it occurs
ZeroDivisionError	जब किसी संख्या को 0 से divide किया जाता है।
ValueError	जब function को सही type का value नहीं मिलता।
TypeError	जब wrong type का operand दिया जाता है।
IndexError	जब list/tuple के valid range से बाहर index access किया जाता है।
KeyError	जब dictionary में non-existent key access की जाती है।
FileNotFoundError	जब specified file नहीं मिलती है।
PermissionError	जब file या resource के लिए permission नहीं होती।
AttributeError	जब object का कोई attribute मौजूद नहीं होता।

5. Key Points

- Exception program के runtime पर occur होती है।
- सभी exceptions "BaseException" class से derive होती हैं।
- try-except का use करके exceptions को handle किया जाता है।
- Multiple except blocks का use करके different exceptions को handle किया जा सकता है।
- अगर exception handle नहीं की जाती, तो program terminate हो जाता है।
- Exception hierarchy हमें सही exception class choose करने में मदद करती है।

Conclusion :

Exception Model और Exception Hierarchy, Python में robust और error-free program लिखने के लिए बहुत important है। इनकी मदद से हम runtime errors को gracefully handle कर सकते हैं और program के crash होने से बचा सकते हैं।

Topic : Handling Multiple Exceptions

- किसी program में एक से अधिक (multiple) exceptions आ सकती हैं, इसलिए हमें उन्हें properly handle करना होता है।
- Python में हम multiple except blocks का use करके different types की exceptions की handle कर सकते हैं।
- यह program की crash होने से बचाता है और हर error के लिए appropriate message देता है।
- Multiple exceptions की handle करने के लिए try-except structure का use किया जाता है।

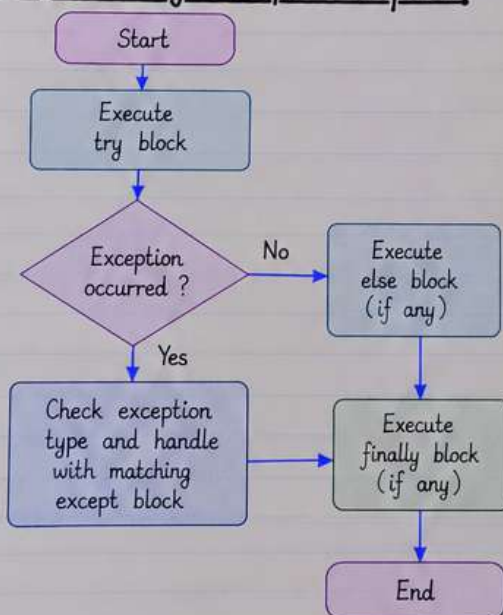
1. Syntax

```
try:
    # code that may raise an exception
    statement(s)
except ExceptionType1:
    # handle exception 1
    statement(s)
except ExceptionType2:
    # handle exception 2
    statement(s)
    :
except ExceptionTypeN:
    # handle exception N
    statement(s)
else:
    # (optional) run if no exception occurs
    statement(s)
finally:
    # (optional) always runs
    statement(s)
```

2. Example : Handling Multiple Exceptions

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    result = a / b
    print("Result = ", result)
except ValueError:
    print("Invalid input ! Please enter an integer.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
except Exception as e:
    print("An unexpected error occurred :", e)
else:
    print("Program executed successfully.")
finally:
    print("This block will always run.")
```

3. Flow of Handling Multiple Exceptions



4. Common Exceptions

Exception Class	When it Occurs
ValueError	Invalid value is provided (e.g. converting non-integer to int)
ZeroDivisionError	Division by zero is performed
TypeError	Operation is performed on incompatible types
FileNotFoundError	Specified file is not found
IndexError	List index is out of range
KeyError	Dictionary key is not found
AttributeError	Invalid attribute access
IOError / OSError	Input/Output related error

6. Key Points

- Multiple except blocks का use करके different exceptions को handle किया जाता है।
- Exception handling se program robust और stable बनता है।
- Specific exception classes को पहले लिखना चाहिए, और general Exception को बाद में।
- else block तब execute होता है जब कोई exception नहीं आती।
- finally block हमेशा execute होता है चाहे exception आए या न आए।
- Proper error messages देने से debugging और user experience बेहतर होता है।

5. Example : User Input with Multiple Exceptions

```
try:
    num = int(input("Enter a number: "))
    print("You entered:", num)
except ValueError:
    print("That is not a valid integer.")
except KeyboardInterrupt:
    print("Input cancelled by user.")
except Exception as e:
    print("Some error occurred :", e)
```

Conclusion :

Handling multiple exceptions is an important concept in Python जो program को unexpected errors से बचाता है और उसे gracefully handle करने में मदद करता है।

Topic : Raise , assert

- Python में raise और assert दोनों का use exceptions के साथ होता है।
- raise का use किसी specific exception को manually trigger (raise) करने के लिए किया जाता है।
- assert का use किसी condition (expression) की जाँच करने के लिए किया जाता है, जो False होने पर AssertionError raise करता है।
- ये दोनों program को error handling और debugging में मदद करते हैं।

1. raise Statement

- raise statement का use किसी exception को explicitly raise (trigger) करने के लिए किया जाता है।
- हम built-in exception या user-defined exception को raise कर सकते हैं।
- raise के साथ exception class और optional error message दिया जाता है।
- यह तब useful होता है जब हमें किसी invalid condition को detect करके खुद से exception generate करना हो।

Syntax :

```
raise ExceptionClass("error message")  
(message optional है)
```

2. Examples of raise

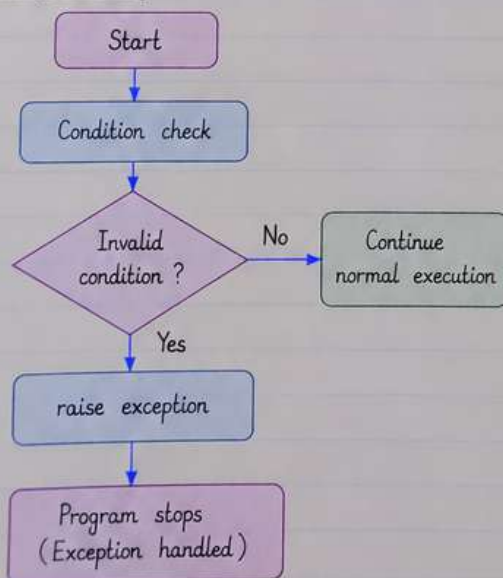
Example 1 : using built-in exception

```
age = -5  
if age < 0:  
    raise ValueError("Age cannot be negative")  
print("Age is :", age) # Age cannot be negative
```

Example 2 : using user-defined exception

```
class InvalidMarksError(Exception):  
    pass  
marks = 150  
if marks > 100:  
    raise InvalidMarksError("Marks cannot be greater than 100")  
# Marks cannot be greater than 100
```

3. Flow of raise



4. assert Statement

- assert statement का use किसी condition (expression) को test करने के लिए किया जाता है।
- अगर condition True होती है, तो program normal रूप से continue करता है।
- अगर condition False होती है, तो AssertionError raise होता है।
- यह mainly debugging और development के समय useful होता है, ताकि logical errors को जल्दी पकड़ा जा सके।
- assert का use program की correctness को verify करने के लिए किया जाता है।

Syntax :

```
assert condition, "error message"  
(message optional है)
```

5. Examples of assert

Example 1 : simple assert

```
age = 18  
assert age >= 0, "Age cannot be negative"  
print("Age is valid :", age) # Age is valid : 18
```

Example 2 : assert with condition

```
x = 10  
y = 0  
assert y != 0, "y should not be zero"  
result = x / y  
print("Result :", result)  
# AssertionError: y should not be zero
```

6. Difference between raise and assert

Point	raise	assert
Purpose	Manually trigger an exception	Test a condition (mainly for debugging)
Exception Type	Any exception (built-in or user-defined)	Always raises AssertionError
Condition	Used when an error condition is detected	Used to check if a condition is True
Use Case	For proper error handling in program	For internal checks during development
Message	Optional	Optional
Best Practice	Used in actual program logic	Used for debugging (not for user input validation)

7. Key Points

- raise का use custom error handling के लिए करें।
- assert का use logical conditions को verify करने के लिए करें।
- assert statements को production code में कम use करना चाहिए।
- raise से हम किसी भी exception class को raise कर सकते हैं।
- assert सिर्फ AssertionError raise करता है।
- दोनों का सही use program को robust और error-free बनाता है।

Topic : in one Example full unit coding.

(Exceptions : Errors, Runtime Errors, Exception Handling, raise, assert)

- इस example में हम अब तक पढ़े गए पूरे unit (Exceptions) के सभी important concepts को एक program में use करेंगे ताकि complete flow समझ आ सके।
- इस program में हम different types की errors को handle करेंगे, raise और assert का use करेंगे, और multiple exceptions को manage करेंगे।
- यह एक realistic example है जो user से input लेकर कुछ operations perform करता है।

1. Problem Statement

User से दो numbers लेना है और उन पर division, list access और file reading जैसी operations perform करनी हैं। Program में सभी possible errors को handle करना है, validate करने के लिए assert use करता है और custom exception raise करती है।

2. Complete Python Program

```
# Full Unit Example : Exception Handling
# with Errors, Runtime Errors, raise and assert

class NegativeNumberError(Exception):
    """Custom exception for negative numbers"""
    pass

def process_numbers():
    try:
        # Taking input from user
        a = int(input("Enter first number: "))
        b = int(input("Enter second number: "))
        # Using assert to validate input
        assert a >= 0 and b >= 0, "Numbers must be non-negative"
        # Custom exception using raise
        if a < 0 or b < 0:
            raise NegativeNumberError("Negative number entered!")
        # Division operation (may raise ZeroDivisionError)
        result = a / b
        print("Division Result:", result)
        # List access (may raise IndexError)
        nums = [10, 20, 30]
        index = int(input("Enter index to access list (0-2): "))
        print("List value:", nums[index])
        # File reading (may raise FileNotFoundError)
        filename = input("Enter filename to read:")
        with open(filename, "r") as f:
            data = f.read()
            print("\nFile Content:")
            print(data)
    except ZeroDivisionError:
        print("Error: Cannot divide by zero.")
    except IndexError:
        print("Error: Invalid index! Please enter 0, 1 or 2:")
    except FileNotFoundError:
        print("Error: File not found! Please check filename.")
    except NegativeNumberError as ne:
        print("Custom Error:", ne)
    except ValueError:
        print("Error: Please enter a valid integer.")
    except AssertionError as ae:
        print("Assertion Error:", ae)
    except Exception as e:
        print("An unexpected error occurred:", e)
    else:
        print("\nAll operations completed successfully.")
    finally:
        print("\nProgram finished. (finally block always runs)")

# Calling the function
process_numbers()
```

3. Sample Run (Output)

```
Enter first number: 10
Enter second number: 2
Division Result : 5.0
Enter index to access list (0-2): 1
List value : 20
Enter filename to read: demo.txt

File Content:
Hello Python!
This is a sample file.

All operations completed successfully.
Program finished. (finally block always runs)
```

Example : When error occurs

```
Enter first number: 10
Enter second number: 0
Error: Cannot divide by zero.
Program finished. (finally block always runs)
```

Example : Negative number (raise)

```
Enter first number: -5
Enter second number: 3
Assertion Error: Numbers must be non-negative
Program finished. (finally block always runs)
```

4. Explanation

- **try block** : Risky code (जैसे input, division, file handling) को execute करता है।
- **assert** : Input की validity check करने के लिए use किया (अगर condition False हूँ तो AssertionError आता है)।
- **raise** : Custom exception (NegativeNumberError) को manually trigger करने के लिए use किया।
- **Multiple except blocks** : Different errors (ZeroDivisionError, IndexError, FileNotFoundError, ValueError, AssertionError) को अलग-अलग handle किया।
- **else block** : जब कोई error नहीं आता, तब execute होता है।
- **finally block** : यह हमेशा execute होता है, चाहे error आए या न आए।

5. Key Takeaways

- Exceptions program को crash होने से बचाते हैं।
- assert और raise debugging में बहुत useful हैं।
- Multiple exceptions को handle करने से program robust बनता है।
- try-except-else-finally का proper use program को reliable और user-friendly बनाता है।